LLMs, VLAs, Codex & Claude CLI



A two-hour AI bootcamp for beginners: from chatbots to robots that act, and the tools to build them

VSP TECHVERSE
An Institute of AI & Robotics

AGENDA

Our 2 Hours at a Glance

0:00	**Welcome and the big picture**	5 min	Why these topics fit together
0:05	**Part 1 · LLMs**	30 min	Tokens, embeddings, neural network, examples, prompting, limits
0:35	**Part 2 · VLAs**	25 min	Architecture, examples, use cases: robots that see and act
1:00	**Break**	5 min	Stretch and questions
1:05	**Part 3 · Codex**	20 min	ChatGPT Codex in your terminal
1:25	**Part 4 · Claude Code**	20 min	Claude CLI in your terminal
1:45	**Vikki Vision**	10 min	Architecture: LLM / VLA models, camera, mic and speaker
1:55	**Wrap-up and Q&A**	5 min	Summary and homework

BIG PICTURE

How Today's Topics Fit Together

LLM
Understands and writes language

→

VLA
Adds eyes and hands: sees, then acts

→

Robot
Does the real-world job

THE BUILDERS' TOOLKIT **AI coding agents in the terminal (Codex, Claude Code)**
The tools developers use to build software for all of the above.

ROBOTICS EXAMPLE A humanoid robot hears "bring me the red cup". An LLM-style brain understands the sentence, a VLA connects what the camera sees to arm movements, and developers wrote and tested that software using the terminal and AI coding tools.

PART 1

LLMs

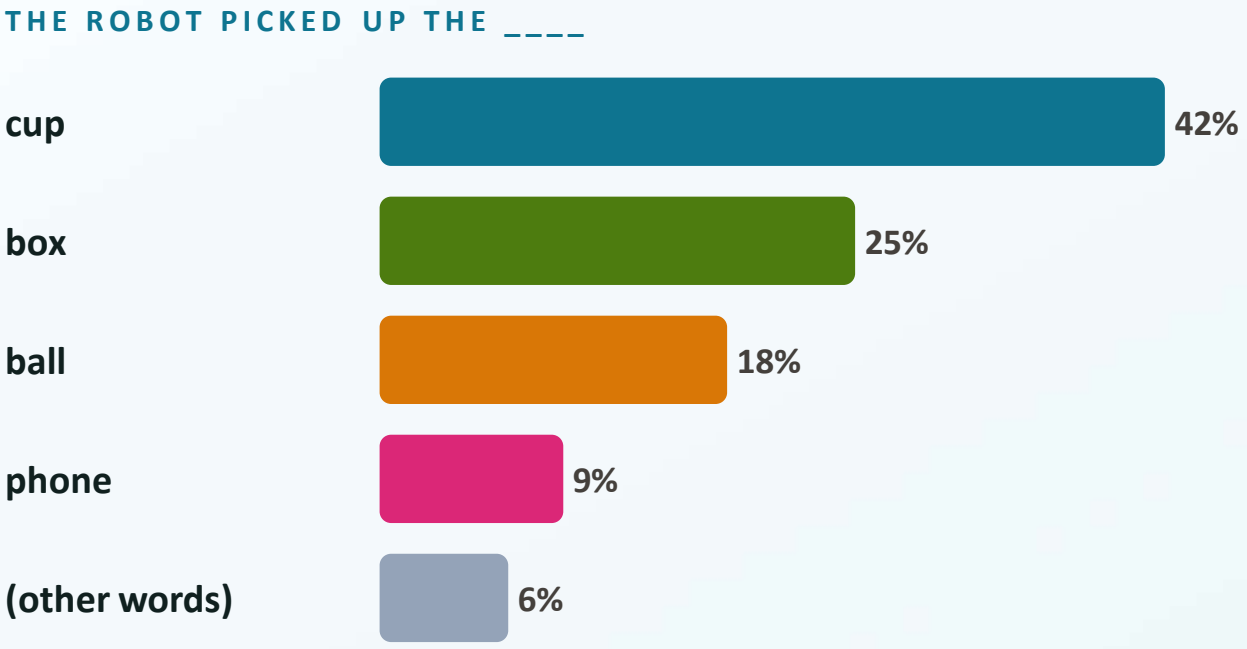
Large Language Models · 30 minutes

- What an LLM really does
- Tokens, training and attention
- Prompting, limits and robots



What Is an LLM? Autocomplete on Steroids

Your phone keyboard suggests the next word. An LLM does the same thing, but it has read a gigantic amount of text and it is a far bigger model.

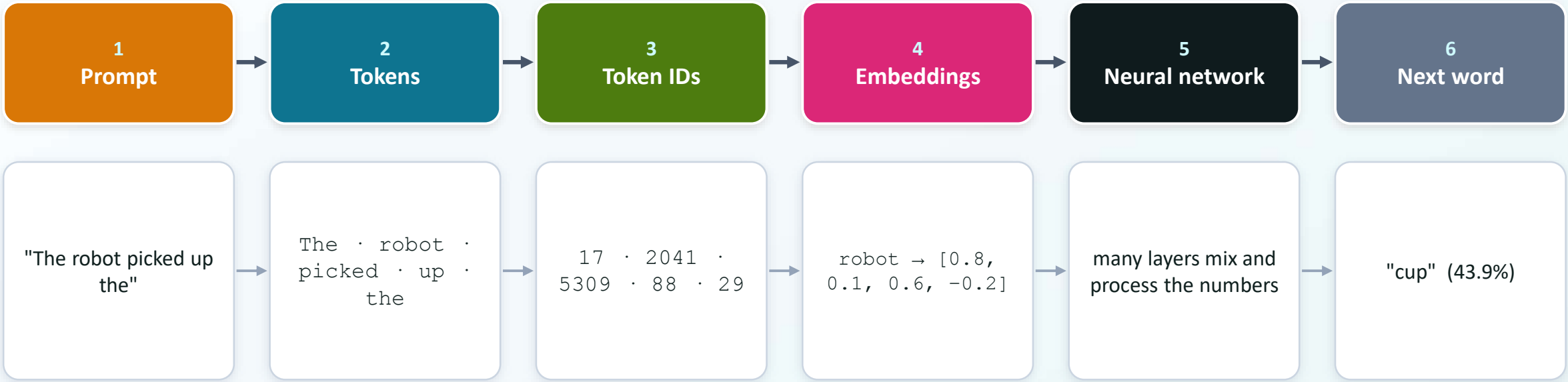


Example numbers for illustration

LLM = Large Language Model

- Large: billions of numbers (parameters) and huge training text
- Language: it works with words and code
- Model: a maths function that predicts what comes next

The Journey: From Prompt to Next Word



Then **"cup"** is added to the prompt and the whole journey starts again to find the next word.

NOTE All numbers on these slides are made-up examples to explain the idea. Real models use much bigger numbers of tokens and values.

Step 1: Prompt → Tokens

```
The robot picked up the
```

TOKENIZER ↓

The	robot	picked	up	the

5 tokens

Why pieces and not whole words?

- The model has a fixed vocabulary of tokens
- Rare or new words are built from known pieces
- Works for code, numbers and many languages

A rare word is split (example)

unbelievable → un | believ | able

Real tokenizers may split differently.

REMEMBER Rule of thumb: 1 token is about ¾ of an English word.

Step 2: Tokens → Token IDs

Token	Token ID
The	17
robot	2041
picked	5309
up	88
the	29

A tiny piece of the vocabulary (example IDs)

Vocabulary = a big dictionary

Every token the model knows has its own number. A real vocabulary holds tens of thousands of tokens or more.

Think of roll numbers in a class

The number is only a label. ID 2041 is not "bigger" or "better" than ID 17. It carries no meaning yet.

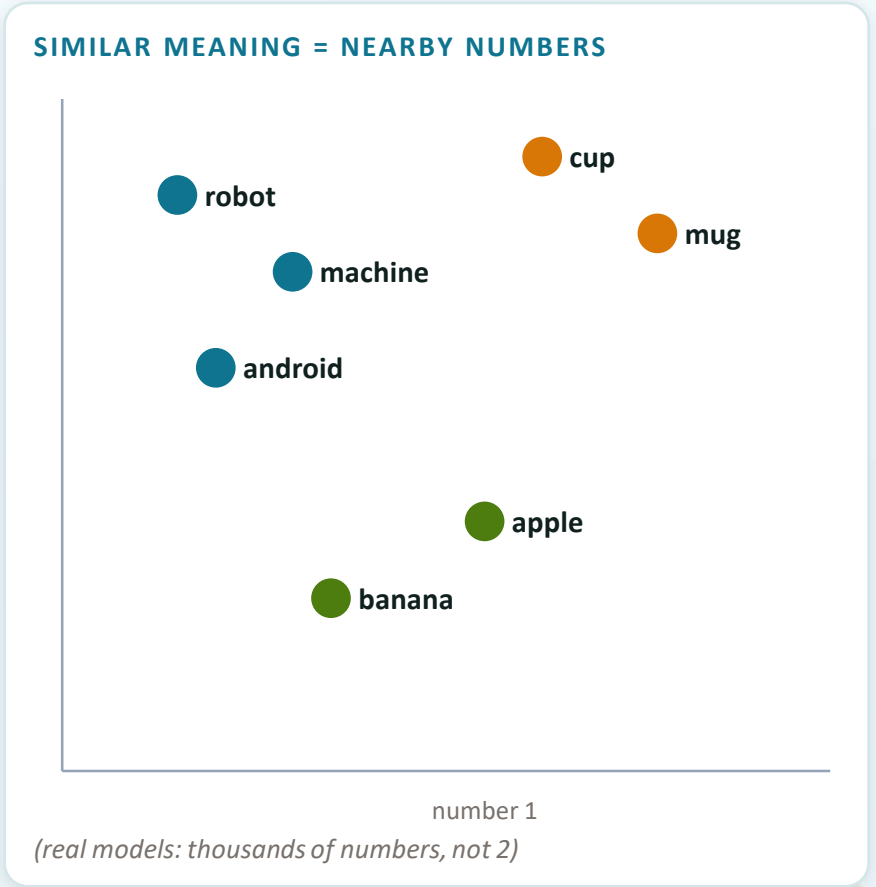
Model input so far: `[17, 2041, 5309, 88, 29]`

WHY Computers only work with numbers, so tokens become IDs. Meaning comes in the next step: embeddings.

Step 3: Token IDs → Embeddings

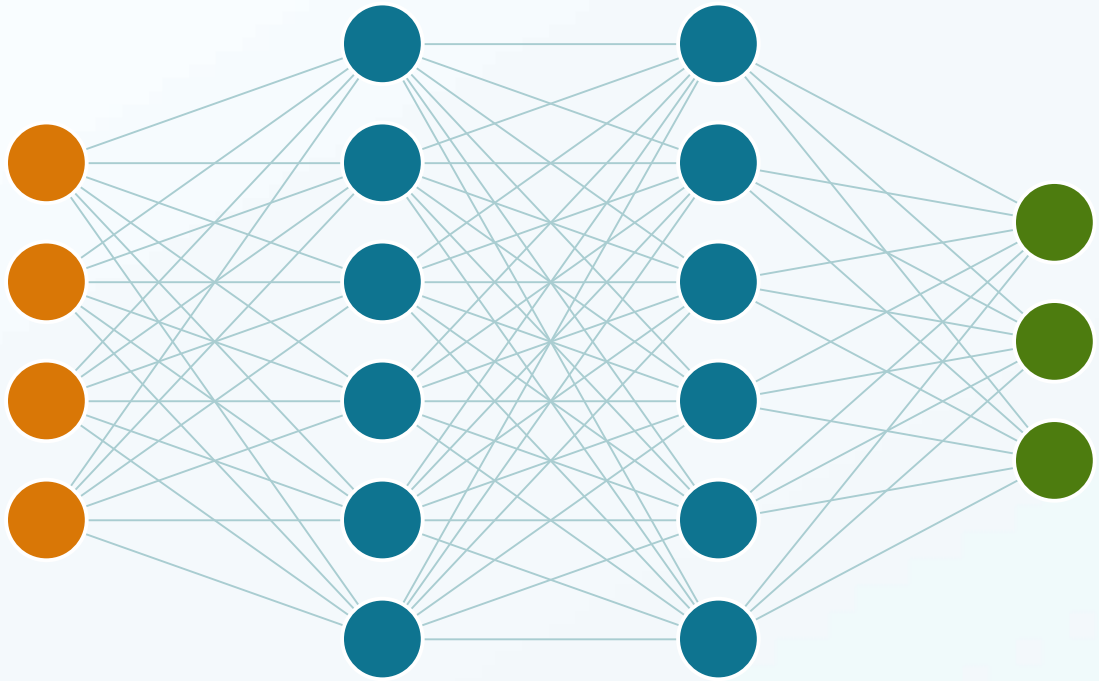
Token	ID	Embedding (4 numbers here)
The	17	[0.1, −0.3, 0.2, 0.0]
robot	2041	[0.8, 0.1, 0.6, −0.2]
picked	5309	[−0.4, 0.7, 0.1, 0.3]
up	88	[0.0, 0.5, −0.6, 0.2]
the	29	[0.1, −0.2, 0.2, 0.1]

Lookup: each ID picks one row from a big table. The numbers are learned during training. Real models use hundreds to thousands of numbers per token. Position information is added too, so the model knows the word order.



KEY IDEA Embedding = a list of numbers that describes a token. Tokens with similar meaning get similar lists.

Step 4a: What Is a Neural Network?



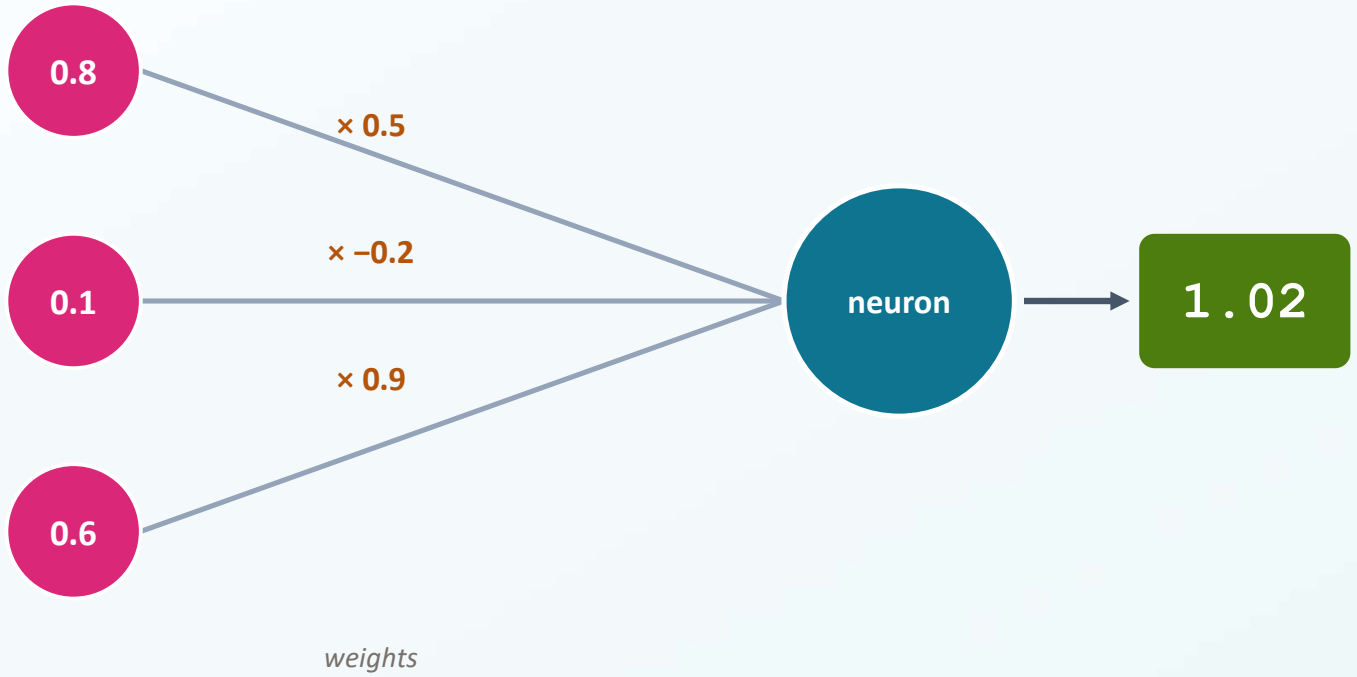
Input numbers | Hidden layers | Output numbers

- Each circle (neuron) does a tiny calculation and passes a number on
- Each line has a weight: a number learned during training
- Our embeddings are the input numbers
- More layers and weights = more powerful. LLMs have billions of weights

KEY IDEA Learning = adjusting the weights again and again until the predictions get better.

Step 4b: One Neuron with Real Numbers

Embedding of "robot" (3 of its numbers)



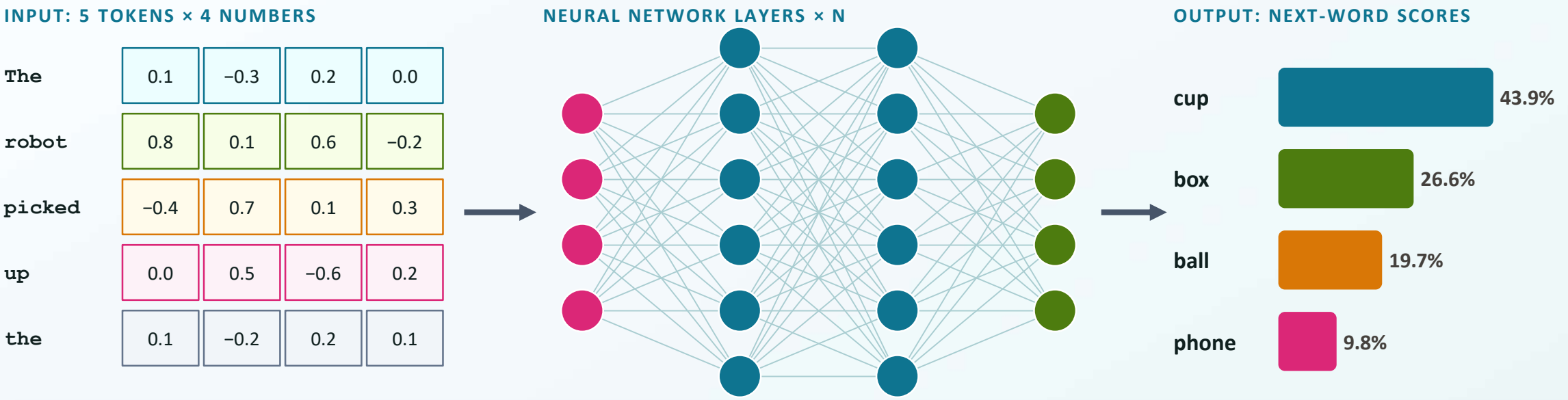
THE CALCULATION

```
0.8 × 0.5    =  0.40
0.1 × −0.2   = −0.02
0.6 × 0.9    =  0.54
sum          =  0.92
+ bias 0.1   =  1.02
```

activation: keep positive numbers, turn negatives into 0 → 1.02

KEY IDEA A layer has many neurons, each with its own weights, so it turns one list of numbers into a new list. An LLM stacks many layers.

Step 4c: Embeddings Through the Network



Inside the layers: attention lets every word look at the others, and neuron layers like the one we calculated transform the numbers. After many layers, the numbers for the last word hold what the model "thinks" should come next, and the output layer turns them into a score for every word it knows (four are shown).

REMEMBER Numbers in, numbers out: the network never sees letters, only the embeddings.

Step 5: Scores → Probabilities → Next Word

Candidate	Score	Probability
cup	3.2	43.9%
box	2.7	26.6%
ball	2.4	19.7%
phone	1.7	9.8%

SOFTMAX: SCORES → PERCENTAGES

$$p = e^{score} \div \Sigma\, e^{scores}$$

The percentages add up to 100%. The highest one (cup) is usually picked.

PICK, ADD, REPEAT

`The robot picked up the →` **cup**

↓

`The robot picked up the cup →` **and**

↓

`The robot picked up the cup and →` **placed**

It stops when the model outputs a special "end" token.

GOOD TO KNOW Temperature: low = always the top word (safe); high = sometimes picks lower words (more creative).

How an LLM Learns: 3 Stages

1. Pretraining

Reads a huge library of text and learns to predict the next word. Like a student who reads thousands of books.

2. Fine-tuning

Practises on examples of good questions and answers. Like exercises and homework at school.

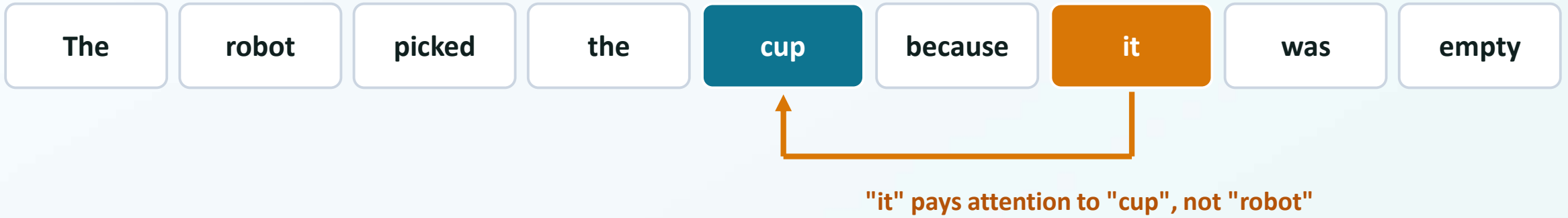
3. Feedback

People (and rules) rate answers, and the model learns to be helpful and safe. Like a teacher's comments.

Training is done once by big companies on powerful computers. We usually use a ready-made model, and we may give it extra information in our prompt.

The Secret Ingredient: Attention

To understand a sentence, the model must decide which words matter for which other words. This is called attention (inside adesign called the Transformer).



Looks at all words

Every word can look at every other word in the text.

Scores importance

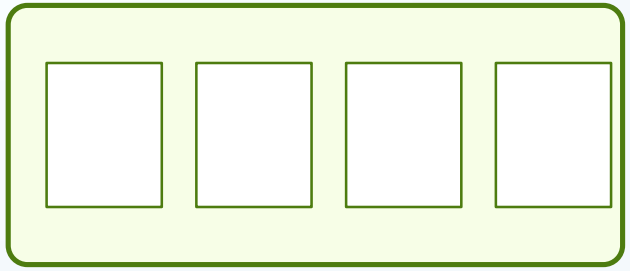
It learns which words matter most for the meaning.

Works in parallel

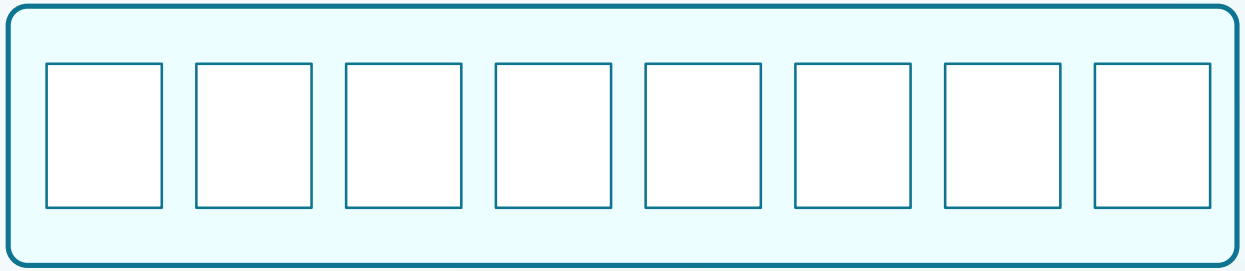
This design trains fast on GPUs, so models could grow huge.

Context Window: The Model's Desk

An LLM can only "see" a limited amount of text at once: its context window. Everything must fit on the desk, your prompt, earlier chat and its answer.



Small desk: A short chat fits. Long documents get cut off or forgotten.



Big desk: Many pages and long conversations fit at once.

PRACTICAL TIP If the chat gets very long, remind the model of the important facts again, or start a fresh chat with a short summary.

LLM Examples and Use Cases

Model	Made by	Known for
GPT	OpenAI	Chat, writing, coding, images
Claude	Anthropic	Writing, analysis, long documents, coding
Gemini	Google	Text, images, audio and video
Llama	Meta	Open-weight: download and run it yourself
Mistral	Mistral AI	Open-weight and hosted models
Qwen, DeepSeek	Alibaba, DeepSeek	Open-weight models you can run yourself

New models appear often, so check the latest versions when you start a project.



Chat and tutoring
Answers questions, explains topics



Coding help
Writes, explains and fixes code



Summaries and translation
Shortens documents, converts languages



Robot voice commands
Understands what a person says

Prompting: Ask Clearly, Get Better Answers

BAD PROMPT

"Write robot code"

Too vague: which robot, which language, what should it do?

GOOD PROMPT

"You are a Python teacher for beginners. Write a function move_forward(distance_cm) for a simple robot that prints each step. Keep it under 15 lines and add comments."

Clear role, task, limits and format.

A SIMPLE RECIPE: R-T-C-F

Role
Who should it act as?

Task
What exactly to do?

Context
What does it need to know?

Format
How should the answer look?

Limits: Never Trust Blindly

Hallucination

It can invent facts, names or code that look real but are wrong.

Old knowledge

It only knows what was in its training data, unless it can search.

Limited memory

Context window is limited, so long chats can be forgotten.

Bias

It can copy unfair patterns from the text it learned from.

Golden rule: check important answers, test every piece of code, and never share passwords, API keys or private data in a prompt.

LLMs Inside a Robot

Voice command

"Bring me the red cup"

LLM plans

1. find cup 2. go to cup 3. pick it 4. bring it

Robot skills

walk, reach, grasp (ready-made programs)

Motors move

Low-level control drives the joints

HIGH-LEVEL vs LOW-LEVEL The LLM runs on the high-level (development) computer. It decides WHAT to do. Low-level compute handles the fast, safe motor control.

The LLM only reads and writes text. How does a robot connect words to what the camera sees and to real movements? That is the next part: VLAs.

QUICK CHECK

Check Yourself: LLMs

1. What does an LLM do, in one sentence, when it writes an answer?

2. What is a token, and what is a context window?

3. Name two limits of LLMs and one safe habit when using them.

ANSWERS 1) It predicts the next token again and again. 2) Token = a piece of a word; context window = how much text the model can see at once. 3) E.g. hallucination, old knowledge, limited memory, bias. Habit: verify answers, test code, keep secrets out of prompts.

PART 2

VLAs

Vision-Language-Action Models · 25 minutes

- From LLM to VLM to VLA
- What is inside a VLA
- Examples, limits and safety



The Ladder: LLM → VLM → VLA

LLM

Input: text

Output: text

Can talk about a cup, but cannot see one.

VLM

Input: images + text

Output: text

Can look at a photo and describe the cup.

VLA

Input: images + text

Output: ACTIONS

Looks at the cup and moves the arm to pick it up.

WORD CHECK VLM = Vision-Language Model. VLA = Vision-Language-Action model: the "A" at the end is what makes a robot move.

What Is a VLA? Eyes + Words → Hands

Camera image
what the robot sees now

Instruction
"Put the red block in the box"

VLA MODEL
one neural network that sees, reads and decides

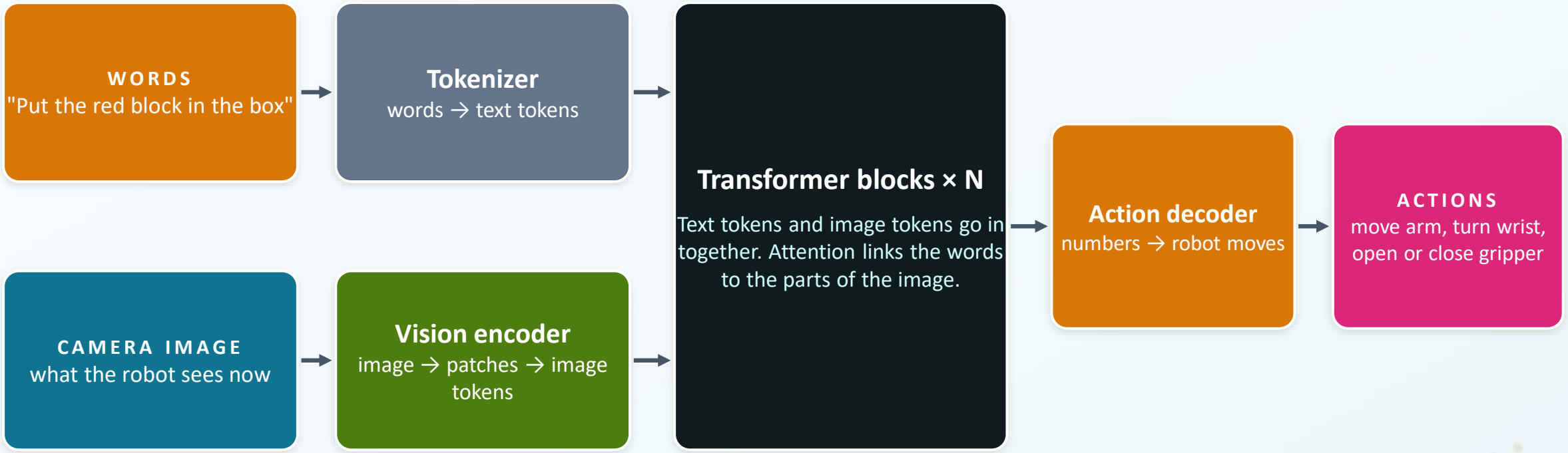
ACTIONS

- move the arm (x, y, z)
- rotate the wrist
- open or close the gripper

It runs in a loop many times per second: look → decide a small movement → move → look again.

ROBOTICS EXAMPLE The model outputs numbers for joints, just as an LLM outputs words. These numbers are often called "action tokens".

VLA Architecture: Words + Images In



Same idea as an LLM, plus **a vision encoder in front** and **an action decoder at the end.**

REMEMBER Analogy: the vision encoder is the eyes, the transformer is the brain, the action decoder is the hands.

How a VLA Learns



Internet images + text

Teaches it what objects are: cups, blocks, tools, and what words mean.



Robot demonstrations

A person controls the robot (teleoperation) while cameras and joints are recorded. The model learns by watching.



Simulation

Practise safely in a virtual world, millions of times faster, before touching real hardware.

Data is the big challenge: collecting good robot demonstrations is slow and costly, much harder than collecting text from the internet.

Well-Known VLA Models

Model	Made by	Why it matters
RT-2	Google DeepMind	Early famous VLA (2023): web knowledge helped robot actions
OpenVLA	Stanford and partners	Open model you can download and study (2024)
Octo	UC Berkeley and partners	Open general-purpose robot policy (2024)
π0 (pi-zero)	Physical Intelligence	General robot model for many tasks (2024)
Gemini Robotics	Google DeepMind	VLA built on the Gemini models (2025)
GR00T N1	NVIDIA	Open foundation model for humanoid robots (2025)

New models appear every few months, so look up the latest ones when you start a project.

VLA Use Cases

Warehouse and factory

Pick, sort and pack items. Assemble parts. Handle many different objects without new code.

Home and service robots

Tidy a table, fetch an object, load a dishwasher when you ask in plain words.

Humanoid helpers

General assistants that follow spoken instructions in offices, shops and hospitals.

Seeing and describing

Look at a scene and explain it aloud, to help people or answer questions. Like Vikki Vision.

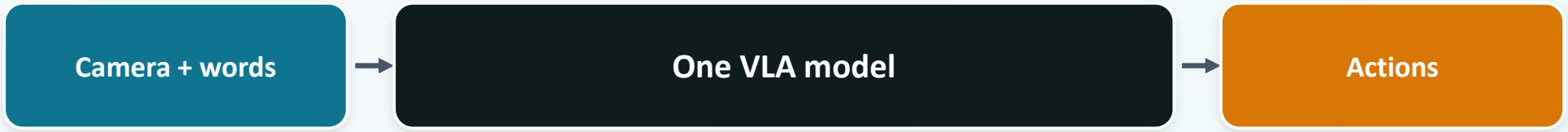
Today: many of these are still being tested in labs and pilot projects. Scene description and question answering with vision models is already easy to build with APIs.

Classic Robot Pipeline vs VLA

CLASSIC: MANY HAND-BUILT PARTS



VLA: ONE LEARNED MODEL



Classic: Easy to test each part and explain failures. But every part needs engineers and may fail on new situations.

VLA: Understands plain English and can handle new objects. But it is harder to explain why it failed.

REAL WORLD In practice, many real robots combine both: learned models plus classic safety checks.

Think Slow, Act Fast

SLOW "THINKING" MODEL

- Understands the goal and the scene
- Runs a few times per second
- Big model, runs on powerful (high-level) compute

FAST "REFLEX" CONTROLLER

- Keeps balance and smooth motion
- Runs hundreds of times per second
- Small model, runs on the robot (low-level compute)

ANALOGY Like you: your brain decides "walk to the door" slowly, while your legs keep you balanced automatically and very fast.

Many humanoid designs split the job this way. It matches the high-level / low-level compute diagram from our bootcamp.

VLAs: Good & Bad

ADVANTAGES

- One model for many tasks
- Follows instructions in plain English
- Can handle new objects and scenes
- Less hand-written rule code

DISADVANTAGES

- Needs lots of robot demonstration data
- Heavy compute (powerful GPUs)
- Hard to explain why it failed
- Safety checks must be added around it

BEST FIT IN ROBOTICS Home and warehouse helper robots, humanoids, robot arms that must handle many different objects.

QUICK CHECK

Check Yourself: VLAs

1. What does the "A" in VLA stand for, and why does it matter?

2. Name the three parts inside a VLA and what each does (eyes, brain, hands).

3. Why is a slow thinking model combined with a fast reflex controller?

ANSWERS 1) Action: it outputs robot movements, not just text. 2) Vision encoder = eyes, language model = brain, action output = hands. 3) Thinking is slow but smart; balance and smooth motion must be fast and safe.

BREAK

5 minutes: stretch, drink water, ask a neighbour one question about the first half.

Next: Codex and Claude Code in the terminal



PART 3

Codex

ChatGPT's coding agent in your terminal · 20 minutes

- What Codex is
- Install, sign in and start
- Commands, approvals and prompts



What Is Codex?

Codex is OpenAI's coding agent. You describe a task in plain English, and it works on your project: reading files, writing code, running commands and fixing mistakes. In the terminal it is called Codex CLI, and you can sign in with your ChatGPT account.



In the terminal

Codex CLI runs inside your project folder, right where your code lives.



In your editor

It can also be used from code editors and other apps.



In the cloud

Tasks can run in the cloud, and you review the result later.



GOOD TO KNOW Claude Code (next part) works in a similar way. Learn the idea once and you can use any coding agent. Exact install steps and features change, so always follow the official documentation.

Chatbot vs Coding Agent

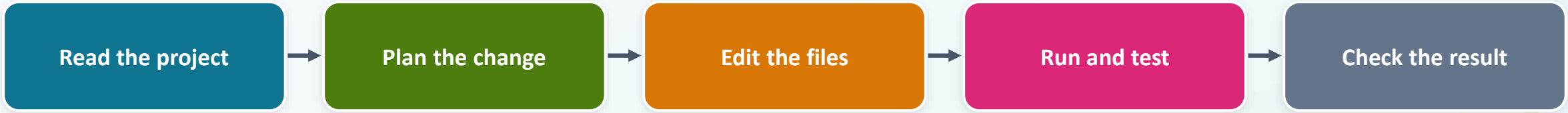
CHATBOT

You copy code from the chat into your files, run it yourself, and paste errors back.

CODING AGENT

It reads your files, edits them directly, runs the code, sees the errors and tries again.

THE AGENT LOOP



KEY IDEA If the test fails, the loop repeats until it works, or until it asks YOU for a decision. You stay the boss: you approve what it does.

Setup Check: Before You Install

Node.js and npm

Both tools install with npm. Check: node --version and npm --version. If missing, install the LTS version of Node.js.

Git

Check: git --version. Git lets you undo anything the agent changes.

An account

Codex: a ChatGPT account (or an API key). Claude Code: a Claude account (or API access). Your trainer will say which to use.

A project folder

Use robot_lab from the CLI part. Run git init inside it so you can go back if needed.

Windows tip: if a tool does not work in PowerShell, use WSL (Windows Subsystem for Linux).

NO ACCOUNT? No account yet? Watch the demo and practise the same commands later.

Install and Start Codex

terminal

```
# 1. Check that Node.js and npm are installed
node --version
npm --version

# 2. Install Codex CLI (only once)
npm install -g @openai/codex

# 3. Go to your project folder
cd robot_lab

# 4. Start Codex, then sign in with your ChatGPT account
codex
```

OUTPUT

```
node --version  → prints a version number
npm install -g ...  → installs Codex (about a minute)
codex  → opens Codex and asks you to sign in
```

Install once

npm install -g @openai/codex downloads the tool. You do this only one time per computer.

Start inside a project

Always run codex from your project folder, so it only works on that folder.

Sign in

On first start choose to sign in with your ChatGPT account. Follow the steps shown on screen.

GOOD TO KNOW Install steps and menus change between versions. Follow the official documentation if something looks different on your screen.

Using Codex: Cheat Sheet

Do this	What happens
`codex`	Start Codex in the current folder
`codex "explain this project"`	Start with a first task or question
`(type a task)`	Describe the goal in plain English and press Enter
`approve / reject`	Read each proposed change and choose on screen
`/`	Type a slash to see commands, such as /init and /model
`Ctrl + C`	Stop what it is doing
`AGENTS.md`	A file of project notes Codex reads, e.g. how to run tests

Approval modes: Codex has modes that decide how much freedom it has, from read-only to full access. While you are learning, keep the careful default so it asks before risky steps.

A Typical Codex Session

terminal

```
# 1. Open the terminal inside your project folder
cd robot_lab

# 2. Start the coding agent there
codex

# 3. Type your task in plain English, for example:
#    Create robot_control.py with move_forward(cm)
#    and turn(degrees) that print what the robot
#    does. Add a small test file too.

# 4. Read the proposed changes, approve them, then run
python robot_control.py
```

Your job: describe the goal clearly, read what the agent proposes, approve it, and test the result yourself.

GOOD TO KNOW Commands and menus differ between versions of the tool, so follow the official documentation for the exact steps on your computer.

Prompts for Coding Agents

BAD PROMPT

"Make robot code better"

No file, no goal, no way to check the result.

GOOD PROMPT

"In robot_control.py, add a function turn(degrees) that prints "Turning N degrees". Do not change other functions. Then create test_robot.py with 3 tests and run them."

Clear file, goal, limit and test.

FOUR HABITS OF A GOOD PROMPT

Goal
What should work at the end?

Where
Which file or folder?

Limits
What must NOT change?

Check
How will we test it?

PART 4

Claude Code

Claude CLI in your terminal · 20 minutes

- What Claude Code is
- Install, log in and start
- Daily commands and Codex vs Claude Code



What Is Claude Code?

Claude Code is Anthropic's coding agent. It runs in your terminal, reads your project, edits files, runs commands and tests, and is powered by Claude models.



In your terminal

Type claude inside any project folder and start talking to it in plain English.



Asks permission

It shows each change and asks before editing files or running commands.



Project notes

A CLAUDE.md file holds instructions, such as how to run the tests and your coding rules.



SAME IDEA It follows the same loop as Codex: read the project, plan, edit, run, check. If you understand one coding agent, you can use the other.

Install and Start Claude Code

terminal

```
# 1. Check that Node.js is installed (version 18 or newer)
node --version

# 2. Install Claude Code (only once)
npm install -g @anthropic-ai/claude-code

# 3. Go to your project folder
cd robot_lab

# 4. Start Claude Code and log in when it asks
claude
```

OUTPUT

```
node --version  → prints a version number
npm install -g ...  → installs Claude Code
claude  → opens Claude Code and asks you to log in
```

Install once

npm install -g @anthropic-ai/claude-code downloads the tool. Do this one time per computer.

Start inside a project

Run claude from your project folder, so it works only on that folder.

Log in

On first start follow the login steps on screen, using your Claude account or API access.

GOOD TO KNOW Other installers exist, and steps change over time. Follow the official documentation if something looks different on your screen.

Using Claude Code: Daily Commands

Type this	What happens
`claude`	Start an interactive session in this folder
`claude "explain this project"`	Start with a first question
`claude -p "what does main.py do?"`	Get one answer, then exit
`claude -c`	Continue your most recent conversation
`/help`	List the available commands
`/init`	Create a CLAUDE.md file with project notes
`/clear`	Start a fresh conversation
`/model`	Choose which Claude model to use

`Esc` interrupts the work

`Shift + Tab` switches permission mode

`Ctrl + C` cancels the current step

A Typical Claude Code Session

terminal

```
# 1. Open the terminal inside your project folder
cd robot_lab

# 2. Start Claude Code
claude

# 3. Type your task in plain English, for example:
#    Explain what hello_robot.py does
#    Add a turn(degrees) function to robot_control.py
#    and a small test file

# 4. Claude shows each change and asks: choose Yes or No

# 5. Run the program yourself
python robot_control.py
```

Your job: describe the goal clearly, read each change Claude proposes, approve or reject it, and test the result yourself.

SAFETY HABIT Never press Yes without reading. If you do not understand a change, ask Claude to explain it first.

Codex vs Claude Code: Side by Side

	Codex	Claude Code
Made by	OpenAI	Anthropic
Install	npm install -g @openai/codex	npm install -g @anthropic-ai/claude-code
Start	codex	claude
Sign in with	ChatGPT account or API key	Claude account or API access
Project notes file	AGENTS.md	CLAUDE.md
Asks before acting	Yes: approval modes	Yes: permission prompts
Works the same way	read → plan → edit → run → check	read → plan → edit → run → check

KEY IDEA Learn the idea, not the buttons: both tools need clear prompts, careful review and testing from you.

Use Agents Safely

Read every change

Review what the agent edited before accepting. You are responsible for the code.

Commit first

Save a git snapshot before a big task, so you can always go back.

Protect secrets

Never put passwords or API keys in prompts, files or commits.

Keep learning

Understand the code. If you cannot explain it, ask the agent to explain it.

Rule for this course: AI can write the first draft, but you must run it, test it and be able to explain every line.

Mini Lab: A Tiny Robot Simulator

Goal: build a small Python program where a robot moves on a grid and avoids obstacles, with Codex or Claude Code as your helper.

1. **Plan in plain words:** Write 3 lines: the grid size, what the robot can do, where the obstacles are.
2. **Ask for step 1:** Prompt your agent to create the grid and print the robot's position.
3. **Ask for step 2:** Add commands forward, left and right. Add a test.
4. **Review and run:** Read the code, run the tests, commit with git.

BONUS Stretch goal: ask the agent to explain its own code line by line.

Check Yourself: Codex and Claude Code

1. What is the difference between a chatbot and a coding agent?

2. What command starts Codex, and which starts Claude Code?

3. Name two safe habits when using a coding agent.

ANSWERS 1) A coding agent reads and edits your files, runs the code and repeats until it works; a chatbot only replies in text. 2) codex and claude. 3) Read every change, commit first with git, protect secrets, understand the code.

Vikki Vision: A Talking, Seeing Robot

A project at VSP AI & Robotics, built in Python. People talk to the robot, and it answers by voice, including describing whatits camera sees.

You speak

The microphone listens to your voice instruction.

It looks

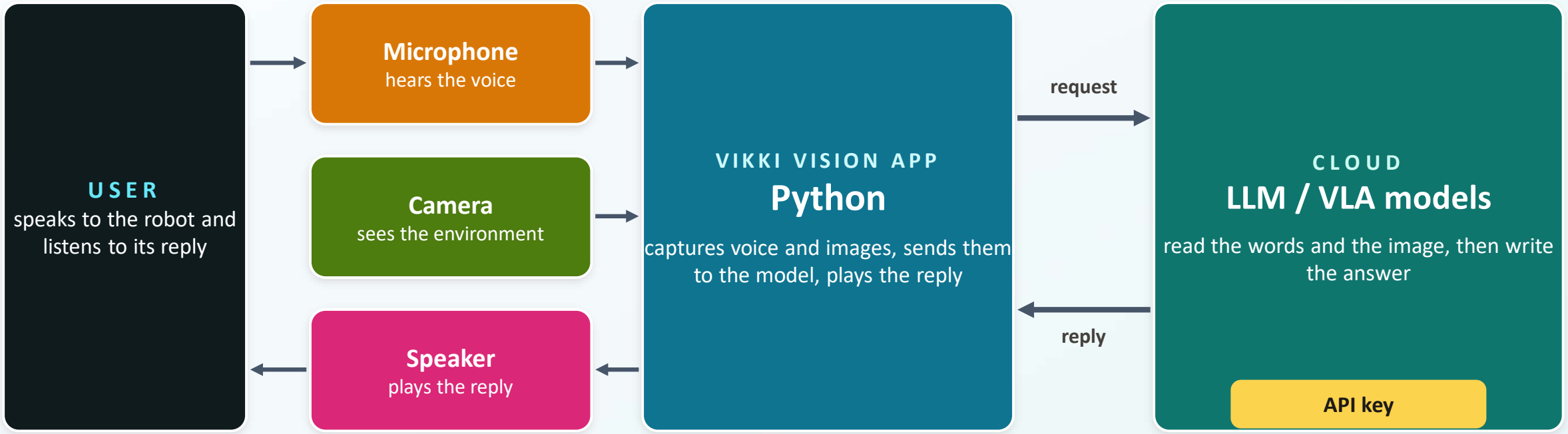
The camera sees objects and the environment around the robot.

It answers aloud

LLM / VLA models understand your words and the image, and the reply is played on the speaker.

EXAMPLE CONVERSATION You: "Vikki, what is in front of you?" Vikki Vision: "I can see a laptop, a cup and a notebook on the table." (example)

Vikki Vision: Architecture



Request: voice instruction + camera image + API key. **Reply:** the answer, played on the speaker.

SAFETY Keep the API key secret: store it safely, never paste it in code you share or in a git commit.

How It Works, Step by Step

1 You speak

The microphone captures your instruction.

2 Camera looks

The camera captures what the robot sees.

3 Python packs it

The app puts your instruction and the image into one request.

4 API call

The request goes to the LLM / VLA model, using the API key.

5 Model thinks

It understands your words and the image and writes the answer.

6 Robot replies

The app plays the answer on the speaker.

WHERE TODAY'S IDEAS APPEAR

LLM
understands your question and writes a natural answer

VLA / vision model
understands what is in the camera image

Python + API key
connects the hardware to the models

SUMMARY

What We Learned Today

LLMs

Predict the next token. Learn in 3 stages. Use clear prompts. Verify answers.

VLAs

Eyes + words → actions. Vision encoder, language model and action output.

Codex & Claude Code

Coding agents in the terminal that edit, run and test. You review and stay responsible.

The chain: LLM understands → VLA sees and acts → the robot works. Coding agents help us build it.

Key Words to Remember

Token
A word or piece of a word

Context window
How much text the model sees at once

Hallucination
A confident answer that is wrong

Prompt
The instruction you give an AI

Attention
How words focus on other words

VLM
Vision-Language Model: image + text → text

VLA
Vision-Language-Action: image + text → actions

Teleoperation
A human controls the robot to teach it

Agentic
The AI plans steps and acts, not just answers

Coding agent
AI that edits files, runs code and tests

Homework: Choose Two

LLM	Ask a chatbot the same question with a bad prompt and a good R-T-C-F prompt. Write 5 lines on the difference.
VLA	Pick one model from the VLA table and write half a page: who made it, what it can do, what data it learned from.
Codex or Claude Code	Finish the tiny robot simulator with a coding agent, and write 3 lines on what you checked yourself.

Thank You



Keep asking, keep building, keep testing

VSP TECHVERSE
An Institute of AI & Robotics